

# DSL Compiler

Domain-Specific Language Compiler using Flex, Bison, C & LLVM IR

---

## Compiler Design – Project Report

|                     |                                     |
|---------------------|-------------------------------------|
| <b>Project</b>      | Compiler-Design-DSL                 |
| <b>Subject</b>      | Compiler Design                     |
| <b>Technologies</b> | Flex, Bison, C, LLVM IR, llc, clang |
| <b>Language</b>     | C (GCC)                             |

# Table of Contents

---

1. Introduction
2. Objectives
3. System Architecture
4. Lexical Analysis (Flex)
5. Parser and Grammar (Bison)
6. Abstract Syntax Tree (AST)
7. Symbol Table
8. Semantic Analysis
9. LLVM IR Code Generation
10. Build and Execution Pipeline
11. Sample Outputs
12. Conclusion

# 1. Introduction

---

This project implements a complete, end-to-end compiler for a minimalist Domain-Specific Language (DSL). The compiler demonstrates all classical phases of compiler construction: lexical analysis, syntactic parsing, abstract syntax tree (AST) construction, semantic analysis with a symbol table, and target code generation in the form of LLVM Intermediate Representation (IR). The generated IR is then compiled to a native executable using the LLVM toolchain (llc and clang).

The DSL is intentionally simple, supporting integer variables, arithmetic expressions (addition, subtraction, multiplication, division), assignment statements, and a print statement. This minimal design keeps the focus on the compiler infrastructure rather than language features.

# 2. Objectives

---

- Design and implement a custom DSL with a well-defined grammar.
- Build a Flex-based lexer that tokenises DSL source files.
- Build a Bison-based LALR(1) parser that constructs an AST.
- Implement a hash-table-based symbol table for variable tracking.
- Perform semantic analysis to detect undeclared variable usage.
- Generate valid LLVM IR manually using fprintf (no LLVM C API).
- Compile the IR to a native binary via llc and clang.
- Produce a professional, modular codebase suitable for academic evaluation and GitHub portfolio.

# 3. System Architecture

---

The compiler follows a classic linear multi-pass pipeline. Each phase is encapsulated in its own module (a .h header and a .c implementation), enabling clean separation of concerns.

| Phase             | Module                      | Output               |
|-------------------|-----------------------------|----------------------|
| Lexical Analysis  | lexer.l (Flex)              | Token stream         |
| Parsing           | parser.y (Bison)            | Abstract Syntax Tree |
| Semantic Analysis | semantic.c + symbol_table.c | Validated AST        |
| Code Generation   | codegen.c                   | LLVM IR (.ll file)   |
| Assembly          | llc (LLVM)                  | Assembly (.s file)   |
| Linking           | clang                       | Native executable    |

# 4. Lexical Analysis (Flex)

The lexer is implemented in `lexer.l` using Flex. It reads the source file character by character and groups characters into tokens. The lexer handles the following token categories:

| Token                  | Pattern                | Description             |
|------------------------|------------------------|-------------------------|
| TOK_BEGIN              | begin                  | Program start keyword   |
| TOK_END                | end                    | Program end keyword     |
| TOK_PRINT              | print                  | Print statement keyword |
| TOK_INTEGER            | [0-9]+                 | Integer literal         |
| TOK_IDENT              | [a-zA-Z_][a-zA-Z0-9_]* | Variable name           |
| TOK_ASSIGN             | =                      | Assignment operator     |
| TOK_SEMI               | ;                      | Statement terminator    |
| TOK_PLUS/MINUS/MUL/DIV | + - * /                | Arithmetic operators    |
| TOK_LPAREN/RPAREN      | ( )                    | Grouping parentheses    |

Whitespace and newlines are silently consumed. Single-line comments starting with `#` are ignored. Unknown characters produce a lexer error message. Integer values are converted to `int` via `atoi` and stored in `yylval.ival`; identifier strings are duplicated with `strdup` and stored in `yylval.sval`.

## 5. Parser and Grammar (Bison)

The parser is written in Bison (`parser.y`) and implements an LALR(1) parser for the DSL grammar. Operator precedence is declared using `%left` directives, ensuring that `*` and `/` bind more tightly than `+` and `-`. Each grammar rule has a semantic action that calls a `make_*`() constructor from `ast.c` to build the corresponding AST node.

### Formal Grammar (BNF):

```
program    ::= 'begin' stmt_list 'end'
stmt_list  ::= (empty) | stmt_list stmt
stmt       ::= IDENT '=' expr ';'
           | 'print' expr ';'
expr       ::= expr '+' expr | expr '-' expr
           | expr '*' expr | expr '/' expr
           | '(' expr ')'
           | INTEGER | IDENT
```

## 6. Abstract Syntax Tree (AST)

The AST is defined in `ast.h` as a tagged union (`ASTNode`). Each node carries a `NodeType` enum tag that identifies which union member is valid. This approach is idiomatic C and avoids dynamic dispatch.

| Node Type | Fields | Represents |
|-----------|--------|------------|
|-----------|--------|------------|

|              |                 |                                  |
|--------------|-----------------|----------------------------------|
| NODE_PROGRAM | stmts[], count  | Root: ordered list of statements |
| NODE_ASSIGN  | var_name, expr  | x = expression                   |
| NODE_PRINT   | expr            | print expression                 |
| NODE_BINOP   | left, right, op | Binary arithmetic operation      |
| NODE_NUMBER  | ival            | Integer literal                  |
| NODE_VAR     | name            | Variable reference               |

Constructor functions (make\_number, make\_var, make\_binop, make\_assign, make\_print, make\_program) allocate nodes on the heap using calloc. A recursive free\_ast() function walks the tree and releases all memory. print\_ast() provides an indented tree view for debugging.

## 7. Symbol Table

---

The symbol table (symbol\_table.h / symbol\_table.c) is a fixed-size hash table with 64 buckets. Collisions are resolved by chaining (a linked list per bucket). The hash function is the classic djb2 algorithm.

API: sym\_table\_create() allocates the table; sym\_table\_insert() registers a variable; sym\_table\_lookup() searches for a name and returns its SymEntry or NULL; sym\_table\_dump() prints all entries for debugging; sym\_table\_destroy() frees all memory.

## 8. Semantic Analysis

---

Semantic analysis is implemented in semantic.c using a two-pass strategy over the AST:

Pass 1 (Declaration Collection): Walks every NODE\_ASSIGN node and inserts the target variable into the symbol table. This pre-population ensures that variables declared anywhere in the program body are available for lookup in Pass 2.

Pass 2 (Use Validation): Recursively walks the entire AST. When a NODE\_VAR is encountered, sym\_table\_lookup() is called. If the variable is not found, a semantic error is reported and the error counter is incremented. After both passes, if error\_count > 0 the compiler aborts before code generation.

## 9. LLVM IR Code Generation

---

The code generator (codegen.c) walks the validated AST and emits LLVM IR text to the output .ll file using plain fprintf calls. No LLVM C API or library is required.

### IR Generation Strategy:

- Module header: source\_filename, target datalayout, declare @printf, @fmt format string.
- define i32 @main() with a single 'entry:' basic block.

- Variable allocation: one `alloca i32` per DSL variable, emitted at the top of `@main`.
- Assignment: the RHS expression is evaluated recursively into SSA temporaries; the result is stored with `store i32`.
- Variable load: a `load i32` instruction reads the current value of a variable.
- Arithmetic: `add` / `sub` / `mul` / `sdiv` instructions on `i32` SSA values.
- Print: `getelementptr` to obtain the format string pointer, then call `@printf` with the value.
- Function return: `ret i32 0`.

### Example IR snippet for 'print x + y;':

```
%t0 = load i32, i32* %x, align 4
%t1 = load i32, i32* %y, align 4
%t2 = add i32 %t0, %t1
%t3 = getelementptr inbounds [4 x i8], [4 x i8]* @fmt, i32 0, i32 0
%t4 = call i32 @printf(i8* %t3, i32 %t2)
```

## 10. Build and Execution Pipeline

---

```
# Step 1: Generate lexer and parser
cd src/
bison -d parser.y          # produces parser.tab.c, parser.tab.h
flex lexer.l               # produces lex.yy.c

# Step 2: Compile the compiler
gcc -Wall lex.yy.c parser.tab.c ast.c semantic.c \
    symbol_table.c codegen.c main.c -o compiler

# Step 3: Compile DSL source to LLVM IR
./compiler ../test/sample1.dsl ../output/output.ll

# Step 4: IR -> Assembly
llc ../output/output.ll -o ../output/output.s

# Step 5: Assembly -> Native executable
clang ../output/output.s -o ../output/executable

# Step 6: Run
../output/executable
```

The Makefile target 'make ir' automates steps 1–6.

## 11. Sample Outputs

---

### Sample 1 — Basic arithmetic (sample1.dsl):

```
begin
  x = 10;
  y = 20;
```

```
    print x + y;
    print x * y;
end
```

Expected output:

```
30
200
```

## Sample 2 — More expressions (sample2.dsl):

```
begin
  a = 100;
  b = 25;
  c = a - b;
  print c;
  print a / b;
  print (a + b) * 2;
end
```

Expected output:

```
75
4
250
```

## Sample Error — Undeclared variable (sample\_error.dsl):

```
begin
  x = 5;
  print z + x;  # 'z' not declared
end
```

Expected error:

```
[Semantic Error] Undeclared variable: 'z'
[Semantic] 1 error(s) found.
[Driver] Semantic errors found. Aborting.
```

## 12. Conclusion

---

This project successfully implements a complete compiler pipeline for a minimal but well-defined DSL. All major compiler phases — lexical analysis, parsing, AST construction, symbol table management, semantic analysis, and LLVM IR code generation — are implemented from scratch in C with clean, modular architecture.

The compiler correctly processes valid DSL programs, produces idiomatic LLVM IR, and the generated IR can be compiled to a native executable that produces the expected numerical output. Semantic errors such as undeclared variable usage are detected and reported before code generation.

The project demonstrates that a working compiler can be built with a relatively small amount of code when the design is kept focused and modular. The codebase is structured for readability and

extensibility: adding new statement types, operators, or data types would require localised changes to the lexer, parser, AST, and code generator without affecting the overall pipeline.

---

End of Report